1 Installation Instructions

This section will focus on installing Python 3.11 and *git*, which is the versioning system we will use. There are many ways to set up Python. Here, we will use *Miniconda* since that is a painless way to do it.

Also, if you have never worked with *git* before, we suggest you take a look at this simple guide once you've installed it: https://rogerdudler.github.io/git-guide/

There is also an example in assignment 1 on how to use *git*.

1.1 Windows

1.1.1 Install Miniconda

- Download the Miniconda installer from https://www.anaconda.com/download (scroll down to Miniconda). Usually the 64-bit (x86_64) version is correct; take the Arm64 version for a Windows laptop with an ARM processor. The installer comes with its own Python version. That does not matter: we create a Python 3.11 environment below.
- Run the installer. Choose *Just Me* and do *not* add Miniconda to your `PATH`.
- Open the *Anaconda Prompt* you just installed (search for it in the Windows search bar).
- Run the command `conda --version`. It should output the version of conda.

1.1.2 Install Git

Install *git* from here: https://git-scm.com/install/windows (the default options are fine).

After installation, close and restart your Anaconda Prompt. Run `git --version`; this should show you your git version and verify that the installation worked correctly.

1.2 Linux

1.2.1 Install Miniconda

- Download the Miniconda installer:
  https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
  On an ARM computer, take `Miniconda3-latest-Linux-aarch64.sh` from the same folder instead.
- Open the terminal and `cd` into the folder containing *Miniconda3-latest-Linux-x86_64.sh*.
- Run `bash Miniconda3-latest-Linux-x86_64.sh`. Accept the license and the default location, and answer `yes` when the installer asks whether to initialize conda.
- Close and reopen your terminal window to make sure the changes take effect.
- Test your installation by running `conda --version` to see its version.

1.2.2 Install git

We will use `apt` (Advanced Package Tool), which is available in Debian, Ubuntu and related Linux distributions, to install *git*. For other Linux distributions, you can find the installation instructions here: https://git-scm.com/install/linux

- Run the following commands in your terminal:

```
sudo apt-get update
sudo apt-get install git
```

- Close and reopen your terminal window to make sure the changes take effect.
- Test your installation by running `git --version` to see its version.

1.3 macOS

1.3.1 Install Miniconda

On a Mac with Apple silicon (M1 and later), the easiest way is the graphical installer (Apple menu → About This Mac tells you which chip your Mac has):

- Download the Miniconda `pkg` installer for Apple silicon (`arm64`) from https://www.anaconda.com/download.
- Open the downloaded `pkg` file in the Finder and follow the instructions on screen.
- When installation is complete, open a new terminal and run `conda --version` to see its version. If you get `command not found: conda`, run `/opt/miniconda3/bin/conda init zsh`, then close and reopen the terminal.

OR, on an Intel Mac or if you prefer the command line:

- Download the Miniconda `bash` installer for your Mac from https://repo.anaconda.com/miniconda/:
  - Apple silicon: `Miniconda3-latest-MacOSX-arm64.sh`
  - Intel: `Miniconda3-latest-MacOSX-x86_64.sh` (Anaconda no longer makes new builds for Intel Macs since August 2025; the last installers are still available there.)
- Open the terminal and `cd` into the folder containing the downloaded file.
- Run `bash Miniconda3-latest-MacOSX-arm64.sh` (or `...-x86_64.sh` on Intel). Accept the license and the default location, and answer `yes` when the installer asks whether to initialize conda.
- Close and reopen your terminal window to make sure the changes take effect.
- Test your installation by running `conda --version` to see its version.

1.3.2 Install git

Open the terminal and run `git --version`. If *git* is not installed yet, macOS offers to install the *Command Line Developer Tools*, which include *git*. Confirm, wait for the installation to finish and run `git --version` again. (You can also start this installation with `xcode-select --install`, or run `brew install git` if you use Homebrew.) More options: https://git-scm.com/install/mac

2 Set up your environment

Next we will set up conda with a new environment named `mydlenv` which uses Python 3.11. Run the following commands in your Anaconda Prompt (Windows) or terminal (macOS, Linux):

```
conda --version
conda update conda -y
conda create --name mydlenv python=3.11 -y
conda activate mydlenv
python --version
conda env list
```

`python --version` should output `Python 3.11.x`. The last command should show you all installed environments (including `base` and the activated `mydlenv` environment). This way, you have a separate environment to install everything for this course and avoid possible interference with/from other python projects.

Note: Recent Miniconda versions ask you to accept Anaconda's Terms of Service before they download packages. If conda asks whether you accept the Terms of Service (ToS) for a channel on `repo.anaconda.com`, type `a` (accept) and press Enter. It may ask once for each channel.

Note: Whenever you start the Anaconda Prompt/terminal for this course, run `conda activate mydlenv`

to switch to the correct environment.

3 Run your first code

We have prepared a hello world git repository for you to test your installation.

- If you don't have one yet, create a GitHub account: https://github.com/signup. You need it to register your team (see assignment 1).
- Create a folder to store your code.
- Open the Anaconda Prompt (Windows) or terminal (macOS, Linux), run `conda activate mydlenv` and `cd` into the folder.
- Clone the public repository: `git clone https://github.com/dl-classroom/ex00-helloworld`
- `cd` into it: `cd ex00-helloworld`
- Run `pip install -r requirements.txt` to install the required package.
- Run `python hello.py` and check the output: it should show Python 3.11 and the numpy version.

Logging in to GitHub: this repository is public, but your team's exercise repositories are private, so *git* asks you to log in the first time you clone one. On Windows, *git* opens a window where you sign in to GitHub with your browser. On macOS and Linux, install the GitHub CLI (https://cli.github.com/) and run `gh auth login` once: choose *GitHub.com* and *HTTPS*, answer *Yes* when asked whether to authenticate *git* with your GitHub credentials, and log in with a web browser.

4 Optional: Get an IDE

4.1 PyCharm

It is possible to edit your code in a simple text editor. However, that is a lot harder than using a good tool. One popular IDE is *PyCharm*.

Install *PyCharm* from here: https://www.jetbrains.com/pycharm/download/
Its free features are all you need for this course. (A new installation starts with a 30-day trial of the paid *Pro* features; as a student, you can also get them for free: https://www.jetbrains.com/academy/student-pack/.) Start it and create a new python project.

Now you want to be able to debug in PyCharm, but also still run code in your command prompt. So you have to tell PyCharm to use your existing conda environment:

- Open the settings (File → Settings; on macOS: PyCharm → Settings) and go to *Project: (your project)* → *Python Interpreter*.
- Click *Add Interpreter* → *Add Local Interpreter*.
- Select an *existing* environment of the type *Conda* and choose `mydlenv`. If PyCharm does not find conda, enter the path to the conda executable. By default, it is
  - *Windows*: `C:\Users\(yourname)\miniconda3\Scripts\conda.exe`
  - *macOS*: `/opt/miniconda3/bin/conda` (graphical installer) or `~/miniconda3/bin/conda` (command line installer)
  - *Linux*: `~/miniconda3/bin/conda`
- Confirm changes.

To find out how PyCharm works, try opening the `ex00-helloworld` folder as a project (File → Open). Make sure the project interpreter is set correctly to your `mydlenv` environment.

Then open `hello.py`, right-click into the editor and choose *Run 'hello'*. You can also try setting a breakpoint in the main function by clicking next to a line number, and then choose *Debug 'hello'* to debug the script.

Note: It can be very helpful in the long run to tune the settings to your preferences and learn the important hotkeys to become a fast coder.

4.2 Visual Studio Code

Another popular IDE choice is *Visual Studio Code* (VS Code), which you can also download for free.

Install *Visual Studio Code* from here: `https://code.visualstudio.com/`

VS Code does not support a specific programming language. Instead, it provides a set of generic features which can be specialized by installing plugins. For Python support in VS Code, install the Microsoft Python extension:

- Open Extensions by clicking on the square icon in the left sidebar or by pressing *Ctrl+Shift+X*.
- Search for "Python" and click *Install* on the Python extension from Microsoft.

In VS Code, you can associate each project with a specific Python environment by following these steps:

- Open the command palette by clicking on "View" → "Command Palette" or by pressing *Ctrl+Shift+P*.
- Type `Python: Select Interpreter` and press *Enter*.
- Select your `mydlenv` conda environment from the list.

To get some hands-on experience, open `ex00-helloworld` as a new project by clicking on "File" → "Open Folder". Then associate your Python environment with the project with the steps described above.

VS Code also provides a built-in terminal which you can open by clicking *Terminal* → *New Terminal*. If the setup was successful, your conda environment should be activated automatically.

Run the `hello.py` Python script in the terminal by typing `python hello.py`.

VS Code also provides a Python Debugger extension, that you can use to debug your code. You can read more about it here.

We recommend PyCharm or Visual Studio Code, as these are in our opinion the best you can get for free.

4.3 Code Together

Another good way to code is doing it together with your teammates. VS Code has an extension for this, Live Share. A platform-independent solution would be to meet over Zoom or over BigBlueButton to discuss, but coding together is definitely more efficient than just discussing together.

5 How to hand in written answers

Hand in your answers to all non-coding questions as a *single* PDF named `submission.pdf`: commit and push it to your team's repository. Put only your team name on top (no GitHub usernames, real names, email addresses or matriculation numbers).

5.1 Optional: Using LaTeX

LaTeX is a professional way of creating documents with math in it. In case you want to use it, we provide a template for you: `submission.tex` in your exercise repository, with the questions of the sheet already in place. Compiling it gives you `submission.pdf`.

5.1.1 Installation

First, you will need to install a LaTeX compiler and package manager, e.g. MiKTeX. Make sure to install for your local user only, not for all users (this can cause problems).

Then, you will need some IDE to work in. If you're already working in PyCharm, you can use the TeXiFy-IDEA plugin; in VS Code, the LaTeX Workshop extension. You can also use a standalone LaTeX IDE like TeXworks (all OS).

Here is a Tutorial to help you get started with LaTeX.

5.1.2 Overleaf

Overleaf allows you to collaboratively edit LaTeX documents in real-time, in your browser and without installing anything. Note that its free plan allows only one collaborator per project.

5.2 Doing the math by hand

Another option would be to do the math on a piece of paper and scan it. Please note that we only accept solutions as a single PDF, so make sure to compile all your scans into a single PDF document. You can use Microsoft Word (Windows), LibreOffice (all OS) or any other software you like for that.